component
advantage.
case → which pattern

Week 4 Tutorial Questions

Question 1: Explain in detail the key components of the State design pattern, using a real-world example to illustrate your explanation. (6 marks)

Answer:

- **Context stores a reference to one of the concrete state objects and delegates to it all state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.**
- **The State interface defines a common interface for all concrete states, encapsulating all behavior associated with a particular state.**
- **Concrete States provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.**

In a video player example, we could create a PlayerContext class which will hold a Concrete States. Initially, the PlayerContext could hold StopState. The PlayerContext may have methods such as Play() and Stop(). Both methods will delegates the works to the state class.

The State interface, PlayerState, could define Play() and Stop().

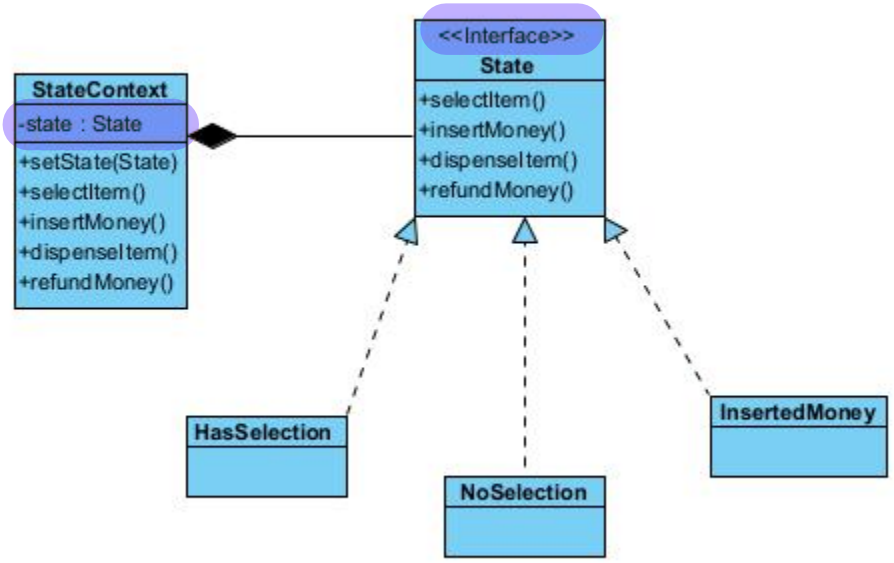
Concrete state such as PlayState and StopState both implement PlayerState. Both state classes implement the methods differently.

Question 2: A vending machine has various states like "NoSelection", "HasSelection", "InsertedMoney", and "Dispensed". The operations (events) a vending machine can perform are selectItem(), insertMoney(), dispenseItem(), cancel(), and refundMoney().

The behavior of these operations changes depending upon the current state of the machine. For instance, you can't dispense an item when there is "NoSelection", or you can't make a selection when the machine is already in the "Dispensed" state. Which is the best design pattern for the vending machine problem? Justify your answer. (6 marks) Draw a class diagram for the design of the system.

Answer: State design pattern is suitable for the problem.

Justification: The vending machine has multiple states. Using State pattern, we can localize state-related operation into state classes making the code simpler and easy to maintain. The operations will transit the vending machine's states, by using State pattern, the state transition can be more explicit and easy to understand.



Question 3: A web server processes incoming HTTP requests through a series of operations, including authentication, logging, and data validation, before the final request is handled. Explain how the Chain of Responsibility pattern can be used to decouple these operations. In your answer, list the key components involved in this design.

Answer:

The Chain of Responsibility pattern decouples the sender of a request from its receivers by giving more than one object a chance to handle the request. Instead of a client knowing exactly which object can process a specific task, it sends the request to the first handler in a chain. The request then travels along the chain until an object handles it or the end of the chain is reached. This allows you to add or change handlers without affecting the client's code.

Key Components:
Handler: An interface or abstract class that defines a method for executing a request and a link to the "next" handler in the chain.
Concrete Handlers: Classes that contain the actual processing logic. They decide whether to process the request or pass it to the next handler.
Client: The component that composes the chain (linking handlers together) and initiates the request to the first handler.

Question 4: The Strategy Design Pattern is often used to implement the Open/Closed Principle (OCP). Using an e-commerce payment system as an example (e.g., Credit Card, PayPal, Bitcoin), discuss how this pattern adheres to OCP and provide two benefits of using this approach in a professional software environment.

Answer:

The Strategy pattern is a direct implementation of OCP. In an e-commerce payment system:

Open for Extension: If the business decides to accept a new payment method (e.g., "Apple Pay"), the developer only needs to create a new ApplePayStrategy class that implements the PaymentStrategy interface.
Closed for Modification: The core PaymentProcessor (the Context class) does not need to be modified or re-compiled to support this new method. It continues to call the generic pay() method on whatever strategy object it holds.

Benefits:
Runtime Flexibility: Algorithms can be swapped at runtime (e.g., a user switching from Credit Card to PayPal in the checkout UI) without restarting the application.
Isolated Unit Testing: Each strategy (e.g., BitcoinStrategy) can be tested in isolation from the rest of the system, ensuring that changes to one payment method do not break others.

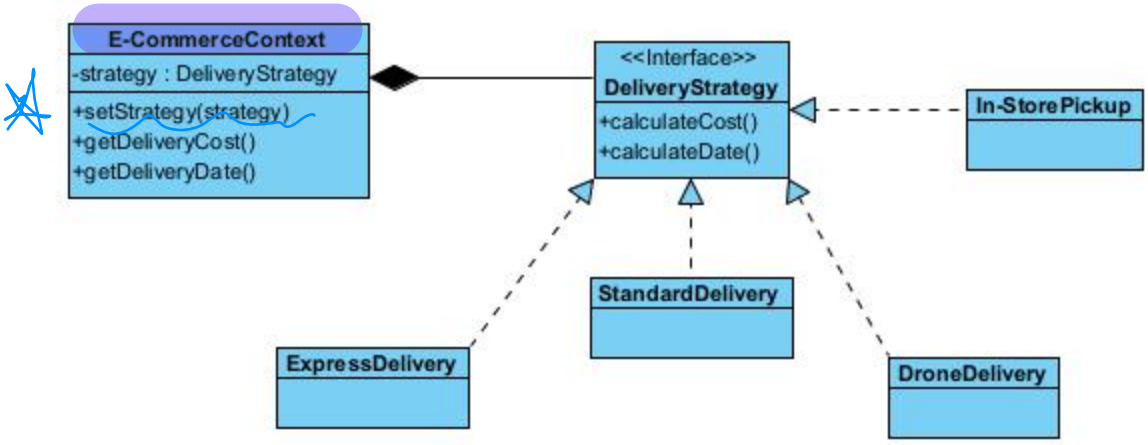
Question 5: An e-commerce platform usually provides multiple delivery strategies for customers such as Express Delivery, Standard Delivery, Drone Delivery, In-Store Pickup and more. Each delivery type has its own set of rules and calculations for cost, delivery date, etc.

1. `Express Delivery`: This could involve a higher cost, but the item will be delivered the same day or the next day.
2. `Standard Delivery`: This typically has a lower cost, but the item might take several days to be delivered.
3. `Drone Delivery`: This involves a high cost, but offers speedy delivery.
4. `In-Store Pickup`: The customer will come to the store to pick up the item, so no delivery charge is applied.

Include all the delivery strategies into one delivery class can clutter the code and lead to challenges for code maintenance. Which design pattern is the best candidate to solve the problem? Justify your answer. (6 marks) Draw a class diagram for the design of the system.

Answer: Strategy design pattern is the best candidate for the problem.

Justification: Each delivery strategy can be encapsulated as a separate class making it more reusable, maintainable, and testable. The delivery strategy is replaceable during runtime making if more flexible,



Question 6: Suggest the scenario when Decorator pattern is appropriate and give a real-world example to illustrate your suggestion. (4 marks)

Answer: Use the Decorator pattern when you need to be able to assign extra behaviors to objects at runtime without breaking the code that uses these objects.

Real-world example, consider the character in a game which can acquire different layers of armors.